

cache copies. Therefore, we see excessive cache coherence overheads, due to false sharing in this code snippet. In order to avoid the false sharing, you need to pad each array element of the array `thread_private_sums` to be present on a separate cache line. I leave it to the reader to make this code change.

A few readers had sent mails asking me to discuss basic lock implementation concepts. So, in this column, we start by looking at mutual exclusion, and some basic algorithms to achieve it.

Mutual exclusion—a primer

Mutual exclusion is ensured by cooperating threads, which follow a uniform locking discipline. Given a critical section of code where mutual exclusion needs to be ensured, a lock is associated with that critical section. Each thread acquires the lock before it can enter the critical section, and releases the lock when it leaves the critical section. Since the lock can be held by only one thread at a time, only one thread can be inside the critical section at a time.

Now readers may ask, “Does not the lock-acquisition code itself constitute a critical section?” The answer to that is, “Yes.” Ensuring mutual exclusion on a critical section is done using locks, but if lock acquisition is itself a critical section, then how do we implement lock acquisition and release operations that require mutual exclusion? Therefore, we first look at some basic lock/unlock algorithms. To simplify it, we will make the assumption that there are only two threads in our system, and the system supports atomic word operations. Consider the following code snippet, which gives a basic lock/unlock operation.

```
boolean flag [2];
void SimpleLock1()
{
    int i = pthread_self();
    int j = 1 - i;
    flag[i] = true
    while( flag[j] ) {} /* just wait until other thread releases the lock */
}
void SimpleUnlock1()
{
    int i = pthread_self();
    flag[i] = false;
}
```

We assume that threads *A* and *B* have IDs 0 and 1 respectively. The Boolean array *flag* represents the intention of a thread to enter the critical section. So if thread *A* with ID 0 wants to enter the critical section, it sets *flag*[0] to true and then checks the value of *flag*[1], which corresponds to thread *B*. If thread *B* is also interested in entering the critical section, and hence has its flag set, then thread *A* just waits until thread *B* finishes the critical section execution. When thread *B* leaves the critical section, it unlocks it by resetting *flag*[1] to false. Thread *A*, polling the *flag*[1] variable, finds that thread *B* has left the critical section, and hence it now enters the critical section. While it is possible to show, with detailed proof, that this algorithm enforces mutual exclusion, we leave it to the reader to derive the actual proof and simply show intuitively why mutual exclusion holds.

For both threads *A* and *B* to be inside the critical section, they should have broken out of the *while* loop in the *SimpleLock1* function. This means that both *flag*[0] and *flag*[1] are false. However, we know that just before entering the *while* loop, the corresponding *flag*[*i*] is set to true by a thread. Hence, both the threads can not be in the critical section at the same time, thereby ensuring mutual exclusion.

What is the issue you see in the above solution? What would happen if both thread *A* and thread *B* set their respective flags to true and come to execute the *while* loop? Each thread would wait forever for the other to reset the flag, leading to deadlock. Is it possible to make this code deadlock-free?

In the above case, when both threads concurrently set their flags to true, deadlock occurs. Consider the code snippet below, where this situation is avoided by using a shared variable which both threads update.

```
void SimpleLock2()
{
    int i = pthread_self();
    sacrificer = i;
    while( sacrificer == i ) {} /* Let the other thread enter critical
section first */
}
void SimpleUnlock2()
{
    //do nothing
}
```

TechnoMail - Enterprise Email Server
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Mobile: 09326087210. info@technoinfotech.com